day12\1_object.js

```
//key is always stored as a string in the object
//to give a space in the key, use double quotes.
//way to create an object
const obj = {
    0: 202,
    1:507,
    2:303,
    3:201,
    name: "yuan",
    age: 20,
    "aadhar number": 4512225002455,
    "instagram id": "ram123"
}
console.log(obj);
console.log(obj.age);
console.log(obj["0"]);
console.log(obj[1]);
console.log(obj[2]);
console.log(obj["aadhar number"]);
console.log(typeof(obj.name));
console.log(typeof(obj.age));
console.log(obj["name"]);
console.log("\n");

//way to create an array
const ary = new Array(10,20,30);
console.log(ary);
console.log("\n");


// the internal implementation of array is based on object concept:
// const ary = {
//     0: 10,
//     1:20,
//     2:30,
//     length: 3
// }
// it means that when we want to access ary.length we don't use the length() function concept because it's a key not a function.


//another way to create an object:
//object is created in the heap
//object is non-premitive data type
const person = new Object();
//making object properties
person.name = "ray";
person.age = 17;
person.gender = "Male";
console.log(person);
delete person.age; //delete object property.
console.log(person);
```

```
person.age = 19; //update age
console.log("\n");



//another way to create an object using class:
//in the class when we declare any variable, then we declare the variable, without the use of any let,var and const keyword.
class people{
    name;
    age;
    gender;
    constructor(namevalue, agevalue, gendervalue)
    {
        this.name = namevalue;
        this.age = agevalue;
        this.gender = gendervalue;
    }
}
//p1 object is created
let p1 = new people("rohit", 20, "male");
console.log(p1);
//p2 object is created
let p2 = new people("fancy", 29, "female");
console.log(p2);


//code snippet
let objecty = {
    name: "jiang",
    age: 30,
    gender: "male"
}
const ar1 = Object.keys(objecty)
console.log(ar1);
const ar2 = Object.values(objecty)
console.log(ar2);
const ar3 = Object.entries(objecty);
console.log(ar3);
console.log("\n");


//code snippet
const ob1 = {a:1, b:2};
const ob2 = {c:3, d:4};
console.log(ob1, ob2);
ob1.a=7;
ob2.a = ob1.a; //this works on the priciple of shallow copy.
console.log(ob2, ob1);
console.log("\n");
//ob1 and ob2 are merged and stored into an empty object and that empty object is get assigned into ob3 object.
const ob3 = Object.assign({}, ob1, ob2); //the 1st parameter that passed is an empty object and other are another objects.
```

```
console.log(ob3, ob2, ob1);
console.log("\n");


//spread operator
const ob5 = {...ob1, ...ob2};
console.log(ob5);

//the spread operator uses shallow copy
const originalobj = {
    //non-nested property-shallow copy
    a: 1,
    //nested property-deep copy
    b: {
    val: 2
    }
};
const copyobj = { ...originalobj };
console.log(copyobj);
originalobj.a = 78;
console.log(copyobj.a);
originalobj.b.val = 514;
console.log(originalobj);
console.log(copyobj);


//read about the topic : shallow copy and deep copy
```